

Existing Auth, Authorization, Identity & Project Model — As-Is Audit

Revision: 1 Last modified: 2026-07-10T11:06:45Z

Scope: a READ-ONLY, evidence-grounded audit of how the Helix OTA control plane (`server/`, Go/Gin modular monolith) authenticates, authorizes, models identity, and models the existing PROJECT concept — so a to-be ACCOUNT (tenant) layer can be designed ABOVE project (account → projects → OTA updates). Every claim below cites a `file:line` actually read. Where the mandate assumes a mechanism that does NOT exist, that is stated plainly with evidence (§11.4.6 — no guessing).

1. Authentication

Scheme: an HMAC-SHA256 signed-opaque bearer token (JWT-*shape* *d*, not a real JWT).

- The token is a two-part `base64url(payload).base64url(HMAC-SHA256(payload))` blob — NOT a standard three-part JWT (`server/internal/api/token.go:72-74` for mint; `:80-89` for the two-part split + constant-time `hmac.Equal` verify).
- Claims carried: `sub` (subject), `roles`, `iat`, `exp` (`server/internal/api/token.go:31-36`). `HasRole` at `:39-46`.
- `TokenSigner` mints (`token.go:60-75`) and verifies (`token.go:77-102`) over a symmetric secret `[]byte` (`token.go:51-58`). It is explicitly documented as the seam the production auth/security brick replaces (`token.go:26-30`, `48-50`).
- Bearer extraction from the Authorization: `Bearer ... header: server/internal/api/middleware.go:99-111` (`bearerToken`); `authMiddleware` verifies and stashes claims in the Gin context under `helix.claims` (`middleware.go:58-77`, key `const middleware.go:14-17`).

A login/sign-in endpoint DOES exist (this is present, not absent):

- `POST /api/v1/auth/login` → `handleLogin` and `POST /api/v1/auth/refresh` → `handleRefresh` are wired as PUBLIC (pre-auth) routes (`server/internal/api/server.go:183-185`).
- `handleLogin` (`server/internal/api/handlers_auth.go:77-99`) exchanges username+password for an access+refresh pair via the injected `UserDirectory` (`s.users.Authenticate`, `handlers_auth.go:93`).
- `issueTokenPair` mints the access token with `cfg.AccessTokenTTL` and issues an opaque refresh token (`handlers_auth.go:123-137`).
- Refresh tokens are random 32-byte opaque strings held in an IN-MEMORY, single-use-rotation, TTL-bounded store (`refreshStore`, `handlers_auth.go:28-72`; `randomOpaque` : `139-148`). Default refresh TTL is a hard-coded 30 days — there is NO config field for it yet (`handlers_auth.go:13-19`).

Where credentials come from — config/env, never the request body's own trust:

- The token signing secret comes ONLY from config: `HELIX_TOKEN_SECRET`, with a development fallback string when unset (`server/internal/config/config.go:66-70` and `:178-184`). The fallback "helix-ota-dev-token-secret-change-me" is a known prod risk (unset env ⇒ predictable signing key).
- Access-token TTL: `HELIX_ACCESS_TOKEN_TTL` / default 15 min (`config.go:30-31`, `54-55`, `153-158`). Device-token TTL: `HELIX_DEVICE_TOKEN_TTL` / default 24 h (`config.go:32-34`, `56-57`).
- The login user directory is seeded from the ENVIRONMENT at process start, never from a request: `HELIX_ADMIN_PASSWORD` (+ optional `HELIX_ADMIN_USERNAME`, default `admin@helix.example`) → one `StaticUser` holding all three roles (`server/cmd/ota-server/main.go:96-104`). An unset admin password disables the static user entirely (`main.go:98`).

Is there an API-key scheme? No (not implemented). The implemented persistence schema has no `api_keys` table (`server/internal/store/schema_postgres.sql`, whole file — none present). An `api_keys` table exists ONLY in the design-target migration (`docs/research/main_specs/1.0.0-mvp/database/migrations/001_initial_schema.up.sql:57-68`), not in the shipped store. Devices authenticate with the SAME signed-token scheme (role device, `token.go:19`), gated on routes like `client/update` / `client/telemetry` (`server.go:223-224`).

2. Authorization

There are **two distinct, independent authorization layers**:

(a) Global RBAC from token roles. Closed role set

admin | operator | viewer | device (server/internal/api/token.go:14-20). requireRole(allowed...) is a per-route middleware that admits a principal carrying at least one allowed role, else 403 (middleware.go:79-97). It is applied on EVERY protected route explicitly — e.g. device register operator/admin (server.go:192), artifact upload operator/admin (server.go:197), audit read admin-only (server.go:242), the full route table server.go:189-251.

(b) Per-PROJECT ACL (a second, finer layer that exists ONLY for project routes). - ProjectRole hierarchy viewer < operator < admin (store.go:297-304; rank compare handlers_project.go:78-92). -

ProjectAccess{ProjectID, CallerID, Role} (store.go:306-311) ties a caller (token subject) to a role within ONE project. - requireProjectAccess (handlers_project.go:37-74) looks up the caller's per-project role via repo.GetProjectAccess and denies below minRole. A GLOBAL admin role bypasses the ACL and implicitly accesses every project (handlers_project.go:43-60; list-all path handlers_project.go:162-199).

What is ABSENT / the load-bearing gap: the per-project ACL is enforced ONLY on /projects/{projectId} GET/PATCH/DELETE (handlers_project.go:204-274). ALL other resources — devices, artifacts, releases, deployments, groups, telemetry, audit — are gated SOLELY by the global requireRole (server.go:192-242); none of them consult project membership, because none of them carry a project id (see §4 and §8). Net effect: **there is no data-level tenant isolation today** — any principal with the global operator/viewer role sees and mutates the entire fleet regardless of any project ACL. The project ACL currently governs project *metadata rows* only.

3. Identity / User model

There is NO persisted User/identity entity in the implemented control plane.

- The store.Repository interface (store.go:315-429) has NO user/identity CRUD of any kind — no CreateUser, GetUser, etc.

- The implemented store schema has NO users table (server/internal/store/schema_postgres.sql, whole file).
- The only “identity” at runtime is the token **Subject string** (token.go:32), an opaque username set at login from StaticUser.Username (main.go:100-101), propagated through the Gin context (middleware.go:74) and read back via getCallerID (handlers_project.go:16-31). ProjectAccess.CallerID IS that bare subject string (store.go:307-311; grant on project-create handlers_project.go:146-154).
- The only identity abstraction is the UserDirectory INTERFACE (server/internal/api/server.go:24-26), implemented by StaticUserDirectory — an in-memory username → {password, roles} map with constant-time password compare (server/internal/api/users.go:8-48), seeded from env (main.go:97-104). There is no sign-up, no persisted per-user record, no user lifecycle.
- Vestige: AuditEntry.UserID exists as a nullable field (store.go:123-134; column schema_postgres.sql:118) intended to key a durable users row, but it is NEVER populated — auditMiddleware sets only ActorSubject (handlers_audit.go:37-46) and toAuditLogEntry falls back to the subject when UserID is empty (audit_wire.go:39-55).

Design-target contrast (valuable for the to-be design): the canonical normalized schema DOES define a users table (id, username, email, role — RBAC subjects) plus api_keys, with FKs from artifacts/releases/deployments/audit back to users (docs/research/main_specs/1.0.0-mvp/database/migrations/001_initial_schema.up.sql:39-50 for users; :57-68 api_keys; FK examples :156, :220, :266, :378). None of that users infrastructure was carried into the shipped schema_postgres.sql (its header explicitly calls itself “the leaner mapping the Repository contract needs”, schema_postgres.sql:1-9). So “where would a User live” has an answer already sketched in the design docs but UNIMPLEMENTED in code.

4. Project concept (as-is)

What a “project” IS today:

- store.Project{ProjectID, Name, Description, CreatedAt, UpdatedAt} (store.go:287-295). Name is UNIQUE. There is NO owner field, NO account field, and — despite the struct’s own doc comment claiming “OS targets, hardware targets” (store.go:287-288) — NO OS/hardware/target fields on the struct at

all. The doc comment is aspirational; the shipped struct is just an id + name + description + timestamps.

- CRUD: `handleCreateProject / List / Get / Update / Delete` (`handlers_project.go:108-274`); wire shape `ProjectResponse` (`wire.go:266-273`).
- Ownership is modeled ONLY through the `ProjectAccess` ACL: the creator is auto-granted per-project admin on create (`handlers_project.go:146-154`); a global admin sees all projects (`handlers_project.go:162-199`).

How projects relate to devices/rollouts/artifacts/deployments: THEY DON'T.

- The `Device`, `Artifact`, `Release`, `Deployment`, `TelemetryRecord`, `Group` structs carry NO `ProjectID` field (`store.go:36-178` — inspect each; none present).
- A repo-wide grep for `ProjectID/project_id` finds hits ONLY in the `Project/ProjectAccess` code paths (`store.go`, `handlers_project.go`, `memory.go:716-850`, `postgres.go:863-974`) — ZERO in `handlers_device.go` / `handlers_release.go` / `handlers_deployment.go` / `handlers_artifact.go` (confirmed: those handlers contain no Project reference).
- The store tables for devices/artifacts/releases/deployments have NO `project_id` column or FK (`schema_postgres.sql:14-70`).

Conclusion: a “project” is an isolated, named metadata row plus a per-project ACL that governs *only that row*. Projects do NOT contain, scope, or partition any OTA resource today. The `account → projects → OTA` updates hierarchy the to-be design wants has its TOP-of-project link entirely missing at the data layer — projects are empty containers.

Namespace note: project Name uniqueness is GLOBAL, not per-tenant (`memory.go:39 prjByName`; `schema_postgres.sql:241 name TEXT NOT NULL UNIQUE`).

5. Persistence seam (`store.Repository`)

Entities the interface persists (`store.go:315-429`): `Projects` + `ProjectAccess` (: 317-332), `Devices` (: 336-340), `Artifacts` (: 343-344), `Releases` (: 347-353), `Deployments` (: 356-360), `Telemetry` (: 363-373), `Audit` (: 376-377), `Rollback history` (: 380-381), `Delta artifacts` (: 384-385),

Groups + members (:390-399), Idempotency keys (:402-403), and the emulation test-fabric registry Nodes/Targets/Leases/Runs/Evidence (:410-428).

Two implementations: MemoryRepository (store/memory.go), wired by DEFAULT (main.go:80); PostgresRepository (store/postgres.go), wired when HELIX_DATABASE_URL is set (main.go:46-78, selection config.go:112-115).

Is the interface additive-friendly for an account/user/membership layer? Yes.

- It is one flat interface; the Projects + ProjectAccess methods (store.go:317-332) were themselves added as a self-contained block and are the exact template to mirror for Account CRUD + AccountMembership + a real User CRUD (which is currently absent — see §3).
- Memory-side attachment mirrors the project maps (memory.go:36-41: projects/prjByName/prjAccess; init memory.go:60-62; impl memory.go:716-850).
- Postgres-side attachment mirrors the projects + project_access tables (schema_postgres.sql:238-258; impl postgres.go:863-974; DDL applied by Migrate postgres.go:49-51).

One caveat for account bootstrap: the seam has NO transaction primitive — the code explicitly notes “there is no cross-call transaction in the Repository seam” and therefore does a best-effort (non-atomic) two-call create-then-grant (handlers_project.go:137-154). An “create account + its first super-admin membership atomically” flow would need either a new transactional method on the seam or the same best-effort compensation pattern.

6. Audit

- auditMiddleware records every SUCCESSFUL (2xx) MUTATING admin/operator action AFTER the handler runs; reads and failed mutations are not audited (handlers_audit.go:21-50). Fields captured: ActorSubject (= token subject), Action (derived SCREAMING_SNAKE verb, :64-93), ResourceType, ResourceID (most-specific path param, :108-116), IPAddress (c.ClientIP()), UserAgent, CreatedAt (:37-46).
- AuditEntry domain struct store.go:123-134; table audit_logs schema_postgres.sql:115-129. Read endpoint handleListAudit

(handlers_audit.go:133-175), ADMIN-only (server.go:242), filterable by action / resource_type / since / until — **no account or project filter exists.**

- **Account-scoping need:** audit rows carry NO account_id/project_id (store.go:123-134, schema_postgres.sql:115-127). To make the audit trail tenant-aware, an account_id (and likely project_id) column + struct field must be added and populated from the resolved account context. Separately, UserID is currently always empty (audit_wire.go:41-43); a real user layer would finally populate it. IP provenance is already hardened (trusted-proxy parsing disabled so ClientIP() is the true peer, server.go:143-159).
-

7. Config & where super-admin bootstrap credentials live

- Config is env-based, fail-fast on malformed values (config.go:118-196); the config brick owns TTLs, limits, base path, secrets-from-env.
 - **The super-admin bootstrap is NOT in config.go — it is in the entryptpoint** (server/cmd/ota-server/main.go:96-104):
HELIX_ADMIN_PASSWORD (+ optional HELIX_ADMIN_USERNAME) mints a single static admin/operator/viewer user; unset password ⇒ no static user. This is §11.4.10-compliant: the credential comes from the environment, never from a request, and is never hard-coded.
 - The signing secret is env-supplied with a dev fallback (config.go:180-184).
 - **Trust-boundary precedent already established in config (important for the account layer):** the code deliberately refuses to derive trust from request headers — the TrustTLSProxy flag is an explicit operator config boolean, NOT inferred from X-Forwarded-Proto (config.go:88-110), and the artifact-signature public key comes ONLY from server config, never the request (resolvePublicKey, handlers_artifact.go:274-283, doc :36). An account super-admin bootstrap must follow the same rule: seed the first account owner from config/env (alongside main.go:96-104 or a new HELIX_* field), never from a self-asserted request.
-

8. Account-layer attachment points (concrete seam list)

To add an ACCOUNT (tenant) layer ABOVE project (account → projects → OTA updates), these seams change:

1. **server/internal/store/store.go:315-429 (Repository interface)** — add an Account block (Create/Get/List/Update/Delete) + AccountMembership (Set/Get/List/Remove) + a real User CRUD (none exists, §3), mirroring the existing Projects block (: 317-332).
2. **server/internal/store/store.go:287-295 (Project struct)** — add AccountID (owning account) so projects hang off an account. New Account struct alongside.
3. **server/internal/store/store.go:36-178 (Device/Artifact/Release/Deployment/ Telemetry/Group structs)** — decide whether OTA resources gain an AccountID (or inherit it via a ProjectID that is added first). Today they carry neither; without one, tenancy isolation is impossible (§4).
4. **server/internal/store/schema_postgres.sql:14-70, 96-129, 238-258** — add accounts + account_members tables (mirror projects/project_access :238-258); add account_id (and/or project_id) FK columns to devices/artifacts/releases/deployments/audit/groups.
5. **server/internal/store/memory.go:36-41, 60-62, 716-850** — add account/membership maps + methods mirroring the project maps.
6. **server/internal/store/postgres.go:863-974** — add account/membership SQL mirroring the project queries; ensure Migrate applies the new DDL (: 49-51).
7. **server/internal/api/token.go:31-36 (Claims)** — decide account/tenant scoping: either add an account_id/tenant claim to the minted token, or resolve membership per-request from the subject. Today the token has roles but NO tenant dimension.
8. **server/internal/api/handlers_project.go:37-74 (requireProjectAccess)** + **server/internal/api/middleware.go:79-97 (requireRole)** — extend to enforce account membership, and add an analogous requireAccountAccess. Wire it onto the currently-unscoped device/artifact/release/deployment/group routes (server.go:192-242), which today have NO tenant gate.
9. **server/internal/api/server.go:183-251 (route table)** — add /accounts CRUD + membership routes; nest project routes under an account.
10. **server/internal/api/handlers_audit.go:21-50 + store.go:123-134 + schema_postgres.sql:115-127** — add account_id to AuditEntry and

populate it (and finally the unused UserID) so the audit trail is tenant-attributable.

11. **server/cmd/ota-server/main.go:96-104 (or a new config.go field)** — bootstrap the first account + its super-admin owner from env/config (§11.4.10 / §7 trust boundary), never from a request.

Single-global-tenant assumptions to flag (each becomes wrong under multi-account):

- **Project name is globally unique** (schema_postgres.sql:241; memory.go:39) → must become unique-per-account.
 - **Device hardware_id is globally unique** (schema_postgres.sql:30; store.go:338 GetDeviceByHardwareID) → likely must become unique-per-account.
 - **Release monotonicity / “latest” is keyed only on (os_type, target_model)** (store.go:349 LatestRelease; the releaseMu invariant server.go:56-69) — global across all tenants; must be scoped per account/project.
 - **Active-deployment uniqueness keyed (os, target_model, group)** (store.go:359 ActiveDeploymentForTarget; deployMu server.go:42-54) — global; must be scoped.
 - **ProjectAccess.CallerID is a bare subject string** with no account qualifier (store.go:307-311) — under multi-account the same subject may belong to several accounts with different roles.
 - **Group name is globally unique** (schema_postgres.sql:102; memory.go grpByName).
 - **Token Claims carry roles but no tenant/account dimension** (token.go:31-36).
-

Files read (provenance)

- server/internal/api/token.go:14-20, 25-36, 39-46, 48-109 — token scheme, Claims, roles, HMAC signer.
- server/internal/api/handlers_auth.go:13-19, 28-72, 77-99, 102-137, 139-148 — login, refresh, refresh store, token-pair issue.
- server/internal/api/middleware.go:14-17, 58-77, 79-97, 99-111, 113-121 — auth middleware, requireRole, bearer extraction, claims context key.
- server/internal/api/server.go:20-26, 28-70, 89-130, 141-256, 258-271 — Server struct, UserDirectory interface, wiring, route table, default policy.
- server/internal/api/users.go:1-48 — StaticUserDirectory / Authenticate.

- `server/internal/api/handlers_project.go:16-31, 33-92, 96-274` — `getCallerID`, `requireProjectAccess`, project CRUD + creator ACL grant.
 - `server/internal/api/handlers_audit.go:21-50, 52-116, 133-175` — audit middleware + list.
 - `server/internal/api/audit_wire.go:9-55` — `AuditLogEntry` / `AuditActor` / `toAuditLogEntry`.
 - `server/internal/api/wire.go:16-36, 245-273` — auth + project wire shapes.
 - `server/internal/api/handlers_artifact.go:36, 126, 274-283` — `resolvePublicKey` trust boundary.
 - `server/internal/store/store.go:22-33, 36-178, 287-311, 315-429` — sentinels, domain structs, `Project/ProjectRole/ProjectAccess`, Repository interface.
 - `server/internal/store/memory.go:34-64, 716-850` — memory fields + project/access impl.
 - `server/internal/store/postgres.go:49-51, 863-974` — Migrate + project/access SQL.
 - `server/internal/store/schema_postgres.sql:1-9, 14-70, 96-129, 238-258` — shipped DDL (no users table, no `project_id` FKs, projects/project_access).
 - `server/internal/config/config.go:16-116, 118-196` — env config, `TokenSecret`, `TrustTLSProxy`, DatabaseURL selection.
 - `server/cmd/ota-server/main.go:33-104, 106-114` — repo selection, admin-user bootstrap from env, server wiring.
 - `docs/research/main_specs/1.0.0-mvp/database/migrations/001_initial_schema.up.sql:35-68, 156, 220, 266, 378` — design-target users + `api_keys` tables + user FKs (UNIMPLEMENTED in the shipped store).
-

Honest gaps

- **Whether an account/tenant concept was ever intended in the shipped MVP:** the `Project` struct doc comment mentions “multi-tenant isolation” (`store.go:287-288`), but no account/tenant table or field exists anywhere in code or shipped schema. I could NOT find any account abstraction — I state its ABSENCE as fact, not the intent behind the absence.
- **PostgreSQL project/access read paths beyond the query text:** I read the SQL strings in `postgres.go:863-974` and confirmed the tables in `schema_postgres.sql`, but did not execute against a live DB;

the “additive-friendly” claim is about the interface shape, not a runtime migration test.

- **Rollout service identity/tenancy:** the staged-rollout service (`internal/rollout/`) is referenced from `server.go:38`, 213-217 but I did not audit its store for any tenant field; on the evidence read it takes deployment ids, not account ids — but I did not open `rollout/store.go`, so I mark this UNCONFIRMED: for the rollout subsystem specifically.
- **Device-token issuance path:** device registration returns a `DeviceToken` (`wire.go:52-59`) and `RoleDevice` exists (`token.go:19`), but I did not read `handlers_device.go` in full, so the exact device-token mint call site is UNCONFIRMED: here (it is not needed for the account-layer attachment analysis, which concerns admin/operator identity).
- **Whether any test wires project-scoped resources:** the audit covered non-test source only (per task scope); a test may exercise a path I did not see. I did not read `*_test.go`.